

Lista de Exercícios — Structs, Ponteiros e Retorno de Dados em C

Parte 1 — Retornando struct

Nesta parte, as funções retornam structs diretamente.

Exemplo:

```
MinhaStruct funcao(...)
```

Sem `malloc`.

Exercício 1 — Criador simples

```
typedef struct{
    char nome[50];
    int idade;
}Pessoa;
```

Faça:

```
Pessoa criarPessoa(char nome[], int idade);
```

A função deve:

- preencher a struct
- retornar a struct

Depois:

- crie 3 pessoas
 - imprima todas
-

Exercício 2 — Soma vetorial

```
typedef struct{
    float x;
    float y;
}Vetor2;
```

Faça:

```
Vetor2 somar(Vetor2 a, Vetor2 b);
```

A função retorna um NOVO vetor.

Teste:

```
(1,2) + (3,4)
```

Exercício 3 — Número complexo

```
typedef struct{
    float real;
    float imag;
}Complexo;
```

Implemente:

```
Complexo multiplicar(Complexo a, Complexo b);
```

Use a fórmula matemática correta.

Exercício 4 — Data futura

```
typedef struct{
    int dia;
    int mes;
    int ano;
}Data;
```

Faça:

```
Data avancarDia(Data d);
```

Ela retorna uma NOVA data com:

- dia +1
- ignore anos bissextos inicialmente

Exercício 5 — RPG

```
typedef struct{  
    char nome[50];  
    int hp;  
    int atk;  
}Personagem;
```

Faça:

```
Personagem buff(Personagem p);
```

A função retorna um personagem com:

- hp + 50
- atk + 10

Sem alterar o original.

Exercício 6 — Coordenada 3D

```
typedef struct{  
    float x,y,z;  
}Ponto3D;
```

Faça:

```
Ponto3D midpoint(Ponto3D a, Ponto3D b);
```

Retorne o ponto médio.

Exercício 7 — Conversão RGB

```
typedef struct{
    int r,g,b;
}RGB;
```

Faça:

```
RGB escurecer(RGB cor, int valor);
```

A função deve retornar uma NOVA cor.

Nunca deixar valores negativos.

Exercício 8 — Produto de loja

```
typedef struct{
    char nome[50];
    float preco;
}Produto;
```

Faça:

```
Produto aplicarDesconto(Produto p, float porcentagem);
```

Retorne o produto modificado.

Parte 2 — Retornando `struct *`

Agora você vai trabalhar com:

```
MinhaStruct *funcao(...)
```

usando:

- `malloc`
 - `free`
-

Exercício 9 — Pessoa dinâmica

Refaça o exercício 1, MAS:

```
Pessoa *criarPessoa(...)
```

Agora:

- use `malloc`
- retorne ponteiro

Depois:

- imprima
 - dê `free`
-

Exercício 10 — Vetor dinâmico

```
typedef struct{  
    int *dados;  
    int tamanho;  
}Vetor;
```

Faça:

```
Vetor *criarVetor(int tamanho);
```

A função deve:

- alocar a struct
- alocar o array interno

Depois:

- preencher
 - imprimir
 - liberar tudo corretamente
-

Exercício 11 — Cadastro de aluno

```
typedef struct{  
    char nome[50];
```

```
float nota;  
}Aluno;
```

Faça:

```
Aluno *novoAluno(char nome[], float nota);
```

Crie vários alunos dinamicamente.

Exercício 12 — Lista de RPG

```
typedef struct{  
    char nome[50];  
    int hp;  
}Monstro;
```

Faça:

```
Monstro *spawnMonstro(char nome[], int hp);
```

Depois:

- crie um array de ponteiros
 - guarde vários monstros
 - imprima todos
 - libere memória
-

Exercício 13 — Matriz dinâmica

```
typedef struct{  
    int linhas;  
    int colunas;  
    int **dados;  
}Matriz;
```

Faça:

```
Matriz *criarMatriz(int l, int c);
```

Depois:

- preencher
- imprimir
- destruir

Exercício 14 — String dinâmica

```
typedef struct{
    char *texto;
}Mensagem;
```

Faça:

```
Mensagem *criarMensagem(char texto[]);
```

Use:

- malloc
- strlen
- strcpy

Exercício 15 — Árvore binária

```
typedef struct No{
    int valor;
    struct No *esq;
    struct No *dir;
}No;
```

Faça:

```
No *novoNo(int valor);
```

Esse exercício é IMPORTANTÍSSIMO.

Você começa a entrar em:

- estruturas de dados reais
- árvores
- sistemas internos

Exercício 16 — Banco simples

```
typedef struct{
    char nome[50];
    float saldo;
}Conta;
```

Faça:

```
Conta *abrirConta(char nome[], float saldoInicial);
```

Depois:

- depósito
- saque
- liberar conta

Exercício 17 — Simulação de OS

```
typedef struct{
    int pid;
    char nome[50];
}Processo;
```

Faça:

```
Processo *criarProcesso(...);
```

Depois:

- mantenha vários processos num vetor
- simule criação e destruição

Exercício 18 — Biblioteca de geometria

Crie structs:

- círculo
- retângulo
- triângulo

Cada função deve retornar ponteiros dinamicamente.

Exercício 19 — Editor de texto simplificado

```
typedef struct{
    char *buffer;
    int tamanho;
}Documento;
```

Funções:

- criar documento
 - escrever
 - destruir
-

Exercício 20 — Mini engine de jogo

```
typedef struct{
    float x,y;
    int hp;
}Entidade;
```

Faça:

- criar entidade
- mover
- dano
- destruir

Tudo usando ponteiros.

Conceitos importantes treinados

Parte 1

Você aprende:

- retorno por valor
- cópia de struct
- stack
- composição de dados
- modelagem de tipos

Parte 2

Você aprende:

- heap
 - ponteiros
 - ownership
 - ciclo de vida
 - gerenciamento manual de memória
 - memory leaks
 - dangling pointers
 - alocação dinâmica
-

Aviso importante

Isso é ERRADO:

```
Pessoa *criar(){
    Pessoa p;
    return &p;
}
```

Porque `p` morre ao sair da função.

Isso é CORRETO:

```
Pessoa *criar(){
    Pessoa *p = malloc(sizeof(Pessoa));
    return p;
}
```

Porque a memória vive no heap.